Efficient Speculative Decoding for Llama at Scale: Challenges and Solutions

GenAI and Infra Teams at Meta. ♡

♡A detailed author list can be found in the Contributions section of this paper.

Speculative decoding is a standard method for accelerating the inference speed of large language models. However, scaling it for production environments poses several engineering challenges, including efficiently implementing different operations (e.g., tree attention and multi-round speculative decoding) on GPU. In this paper, we detail the training and inference optimization techniques that we have implemented to enable EAGLE-based speculative decoding at a production scale for Llama models. With these changes, we achieve a new state-of-the-art inference latency for Llama models. For example, Llama4 Maverick decodes at a speed of about 4 ms per token (with a batch size of one) on 8 NVIDIA H100 GPUs, which is 10% faster than the previously best known method. Furthermore, for EAGLE-based speculative decoding, our optimizations enable us to achieve a speed-up for large batch sizes between 1.4× and 2.0× at production scale.

Date: August 11, 2025
Correspondence: Sachin Mehta at sacmehta@meta.com

Meta

1 Introduction

Attention-based transformers of Vaswani et al. (2017) are widely used across different areas of machine learning and artificial intelligence, including natural language processing, computer vision, and speech processing. Specifically, transformer-based large language models (LLMs) have been scaled to billions of parameters, and are trained on trillions of tokens to produce high-quality models (e.g., Achiam et al., 2023; Grattafiori et al., 2024; Meta, 2025; Team et al., 2023). Because of their auto-regressive nature and large size, the decoding speed fo these models' is very slow, posing significant challenges when deployed in production environments where they must handle a large volume of requests with varying input and output lengths.

Several methods, including FlashAttention (Dao et al., 2022; Dao, 2024), memory-efficient attention Rabe and Staats (2021), fully-sharded data parallel (Baines et al., 2021), and disaggregated inference (Zhong et al., 2024; Qin et al., 2024), have been proposed to improve the training and inference speed of transformer-based models. Complementary to these methods, speculative decoding (Leviathan et al., 2023; Chen et al., 2023) has emerged as a promising technique for accelerating the inference speed of LLMs. Briefly, speculative decoding involves predicting multiple tokens using a smaller model (aka, draft model) auto-regressively, which are validated in a single step using an LLM. This reduces the number of calls to the auto-regressive LLM, improving decoding speed. However, this approach requires significantly more floating point operations (FLOPs) than the non-speculative decoding model because multiple tokens need to be validated by the LLM as opposed to single token decoding in case of a non-speculative decoding setup. Several speculative decoding methods have been proposed (e.g., Cai et al., 2024; Miao et al., 2024; Li et al., 2024). However, most of these methods are benchmarked at small scale settings (e.g., batch size of one) to demonstrate the speed-up over non-speculative decoding method, likely because of significant engineering challenges in scaling these methods to production environments that require handling a large volume of dynamic requests efficiently. For example, when the batch size is increased from 2 to 48, the speed-up (measured using vLLM) of EAGLE-based speculative decoding compared to non-speculative decoding drops from 1.3× to 0.7× (see Table 5 in v3 of Li et al. (2025)). Similar behaviour was also observed with SGLang (SGLang, 2023) (see Table 5 in (Li et al., 2025)).

In this paper, we detail the training (see Section 2) and inference (see Section 3) optimization techniques we have implemented to enable EAGLE-based speculative decoding for Llama models at a production scale, addressing the challenges associated with deploying these models in real-world applications. Figure 1 shows

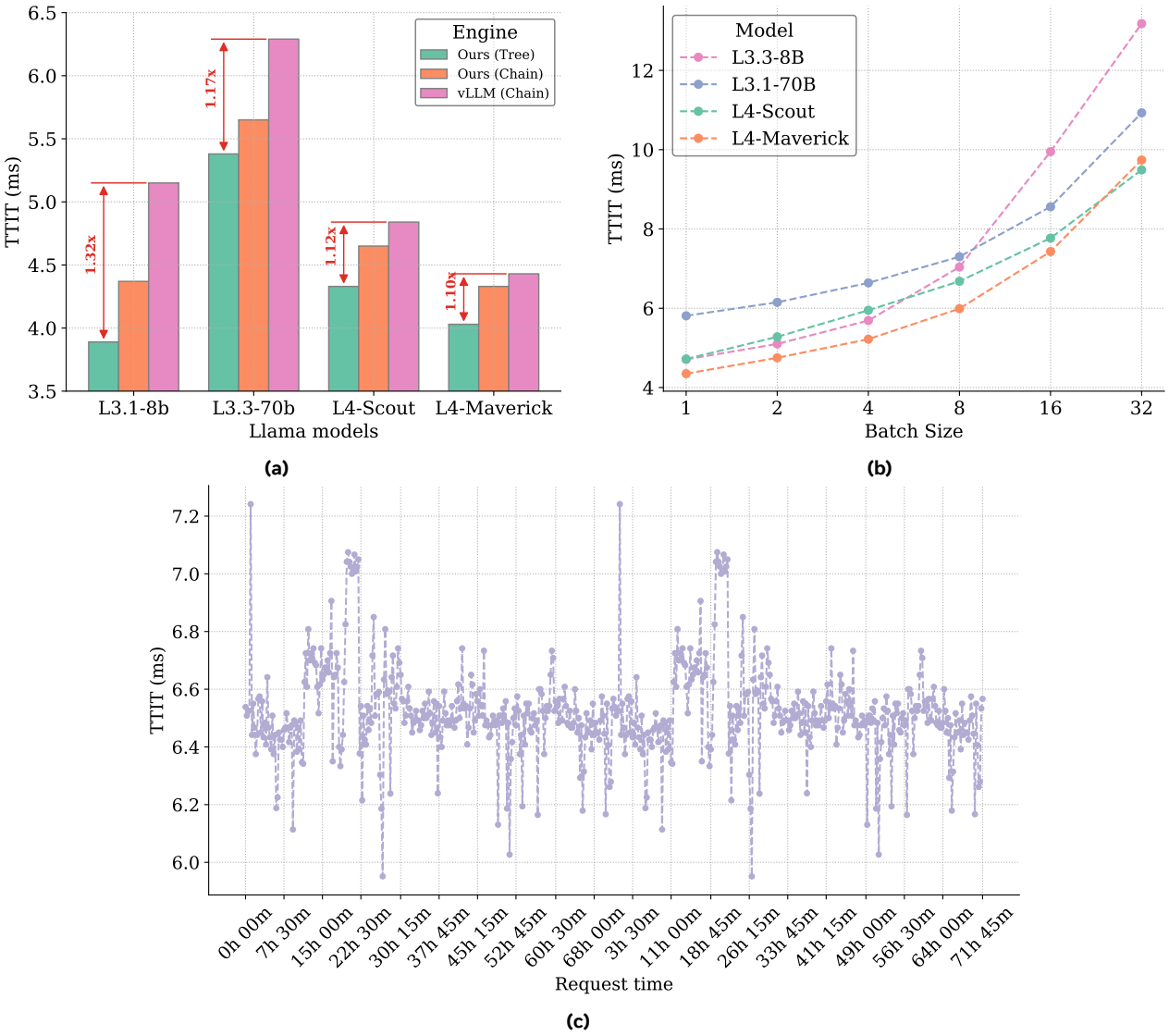


Figure 1 Performance evaluation of our EAGLE-based speculative decoding system. (a) TTIT comparison of our system with vLLM at a batch size of one. (b) TTIT comparison of different LLaMA models within our system at various batch sizes and a context length of 8k. (c) TTIT for online user requests using LLaMA 3.3 70B over a 3-day period. In (b), we do not compare with vLLM because of significant gaps in TTIT between our system and vLLM. This is likely because vLLM stacks might not be optimized for large batch sizes, as also observed in other speculative decoding works (e.g., Li et al., 2025).

that, with our optimizations, the decoding speed (with a batch size of one) of Llama models with EAGLE on 8 NVIDIA H100 GPUs improves by about 10-30% as compared to widely used open-source library vLLM. Moreover, at large batch sizes, our optimizations for EAGLE-based speculative decoding further enable a speed-up between 1.4× and 2×. We note that the proposed optimizations are complementary to open-source libraries (e.g., Kwon et al., 2023; SGLang, 2023) and can be easily integrated to further improve the inference of supported speculative decoding methods.

2 Training optimizations

We introduce three changes for EAGLE-based speculative decoding: (a) online distillation (see Section 2.1), (b) longer training (see Section 2.2), and (c) multi-layer dense draft model (see Section 2.3). With these

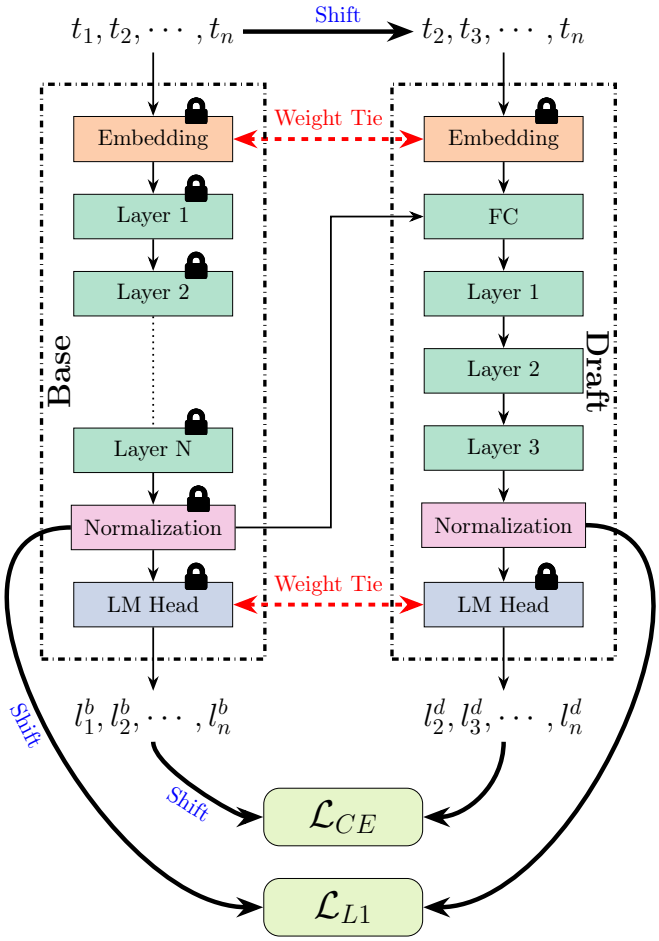


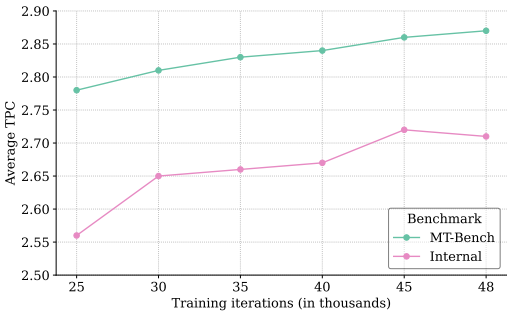
Figure 2 Overview of online distillation that we used to train EAGLE speculative decoding method for Llama-3 and Llama-4 models. Here, 🔒 represents frozen layers.

changes, we trained draft models for four different Llama models that differs in several aspects, including architecture and model size. The results are then presented in Section 2.4.

2.1 Online distillation

An overview of our implementation of EAGLE speculative decoding is given in Figure 2. Specifically, it uses a base model $\mathcal{B}$ and a draft model $\mathcal{D}$, both of which are auto-regressive. The base model consists of N sequential transformer layers, while the draft model is light-weight with M layers, where $M <<< N$. During training, the base model $\mathcal{B}$ takes n tokens $\mathbf{t} = \{t_1, t_2, \cdots, t_n\}$ as input. It produces hidden states $\mathbf{h}^b = \{h_1^b, h_2^b, \cdots, h_n^b\}$ (i.e., output from the normalization layer before LM head) and logits $\mathbf{l^b} = \{l_1^b, l_2^b, \cdots, l_n^b\}$ (i.e., output from the LM head before softmax). For the draft model, the input tokens are shifted to the right by one, resulting in $\hat{\mathbf{t}} = \{t_2, t_3, \cdots, t_n\}$. These tokens are fed into the embedding layer. Unlike the base model, the draft model includes a fully-connected layer between embedding and the first transformer layer. The layer takes the token embeddings of $\hat{\mathbf{t}}$ and hidden states $\mathbf{h}$ from the base model as inputs, and the resulting output is fed to the rest of the draft model to produce the hidden states $\mathbf{h}^d = \{h_2^d, h_3^d, \cdots, h_n^b\}$ and logits $\mathbf{l^d} = \{l_2^d, l_3^b, \cdots, l_n^b\}$.

To train the draft model, we minimize the loss between (a) hidden states and (b) logits of the base $\mathcal{B}$ and the draft $\mathcal{D}$ models. Specifically, we compute the smooth L1 loss between the right shifted hidden states of the base model and the hidden states of the draft model, as $\mathcal{L}_{L1} = \text{l1\_loss}(h_{2:n}^b, h_{2:n}^d)$. Similarly, we compute the cross entropy loss between shifted logits of the base model and those of the draft model, as



(a)

# Layers	Model configurations		TPC ↑	
	iRoPE	FFN Type	MT-Bench	Internal
2	Yes	MoE	2.75	2.55
		Dense	2.79	2.58
	No	MoE	2.75	2.55
		Dense	2.80	2.58
3	No	Dense	**2.87**	**2.71**

(b)

Figure 3 Ablation with Llama4 Scout. (a) Effect of training duration on the Llama4 Scout's TPC on two benchmarks, MT-Bench and internal benchmarks. (b) Effect of different draft model design choices for Llama4 Scout base model. Here, TPC is measured with chain-like draft, temperature=0, top-p=0.9, and speculation length of three. iRoPE denotes interleaved RoPE.

$\mathcal{L}_{CE} = \text{ce\_loss}(l^b_{2:n}, l^d_{2:n})$. The final loss is a weighted sum between $\mathcal{L}_{L1}$ and $\mathcal{L}_{CE}$, as

$$\mathcal{L} = \lambda_{CE} \cdot \mathcal{L}_{CE} + \lambda_{L1} \cdot \mathcal{L}_{L1} \tag{1}$$

where λ_{CE} and λ_{L1} are the coefficients to control the contributions of cross entropy and L1 loss, respectively. In our experiments, we use $\lambda_{CE} = 0.1$ and $\lambda_{L1} = 1.0$.

2.2 Longer training

We train draft models for the following four base models belonging to Llama3 (Grattafiori et al., 2024) and Llama4 (Meta, 2025) families: (1) Llama3.1 8B, (2) Llama3.1 70B, (3) Llama4 Scout (total and active parameters are about 109B and 17B respectively), and (4) Llama4 Maverick (total and active parameters are about 400B and 17B respectively). The models in Llama3 family uses dense feed-forward network (FFN) layers while Llama4 models uses mixture-of-experts (MoE; (Shazeer et al., 2017)) layers. We use the same supervised fine-tuning (SFT) dataset that was used to train the models for a total of 48k iterations, with 2M tokens per iteration. We optimize the draft model using Adam Kingma (2014), with a constant learning rate of 0.0002 and a weight decay of 0.1. To evaluate the quality of the draft model, we measured the number of accepted tokens per drafting-validation stage (or tokens per call; TPC) on two benchmarks: (a) MT-Bench (Zheng et al., 2023), a public benchmark that is widely used for measuring speculative decoding performance, and (2) a private internal benchmark, that contains a diverse, multi-lingual and harder samples. We used a batch size of one and chain-like draft with speculation length of three for TPC evaluation. A higher value of TPC is desirable.

Figure 3a shows the effect of longer draft training using Llama4 Scout. We see that longer training helps improve the average tokens accepted per drafting step or tokens accepted per call (TPC) on both benchmarks. Note that we observe a similar trend in other models as well.

2.3 Multi-layer dense draft models

For the draft model design, we considered transformer blocks with dense and MoE FFN. We also studied the relationship between the number of transformer blocks and the quality metrics. The results are shown in Figure 3b. We make following observations:

- **Dense vs. MoE FFN:** Our results indicate that draft models with dense FFN achieve comparable TPC to MoE, while utilizing substantially fewer parameters. For example, the draft model for Llama4 Maverick with dense FFN layers has about 10× fewer total parameters as compared to MoE. Therefore, we used dense FFN layers in our draft models.
- **Effect of iRoPE:** Llama4 introduced interleaved ROPE (iRoPE). We did not observe any significant differences in TPC or end-to-end latency with and without iRoPE. Therefore, iRoPE was not used in the training of our final draft models.

Model	Speculation length	TPC ↑
Llama3.1-8B w/ EAGLE	3/5/7	2.29/2.44/2.47
Llama3.1-8B w/ EAGLE3	3/5/7	2.80/3.32/3.57
Llama3.1-8B w/ EAGLE	3/5/7	2.12/2.24/2.27
Llama3.3-70B w/ EAGLE3	3/5/7	2.64/3.03/3.20
Llama3.1-8B w/ EAGLE (Ours)	3	2.78
Llama3.3-70B w/ EAGLE (Ours)	3	2.94
Llama4-Scout w/ EAGLE (Ours)	3	2.87
Llama4-Maverick w/ EAGLE (Ours)	3	2.75

Table 1 TPC results for different Llama models on the MT-Bench benchmark. Here, TPC is measured with chain-like draft, temperature=0, and top-p=0.9.

- **Effect of number of layers:** We varied the number of layers from 1 to 3 and observed a notable improvement in TPC, from 2.63 to 2.87. However, further increasing the model's depth beyond 3 layers did not yield significant gains in TPC. Therefore, we choose a 3-layer dense draft model as our final candidate for speculative decoding.

2.4 Results

Results for different Llama3 and Llama4 models on the MT-bench benchmark are shown in Table 1. The TPC values vary across models, with Llama3.3 70B achieving the highest value of 2.94 and Llama4-Maverick achieving the lowest value of 2.75, likely due to differences in model capacity and architecture. As a reference, Table 1 also includes the original results for the 8B and 70B models with both EAGLE and EAGLE3 (Li et al., 2025), measured using vLLM (Kwon et al., 2023) at different speculation lengths. With the proposed changes, EAGLE achieves similar or better TPC than EAGLE3, highlighting the effectiveness of the introduced training optimizations. Interestingly, EAGLE with the proposed changes and a speculation length of three outperforms the EAGLE from Li et al. (2024) even at a speculation length of seven.

3 Inference optimizations

Our EAGLE speculative decoding inference process, shown in Figure 4, is organized into the following stages: (1) *Prefilling:* This stage involves executing the base model prefill on input prompt tokens, followed by the draft model prefill on the input tokens and hidden states produced by the base model. (2) *Tree Dispatcher:* In this stage, an optimal static tree structure is selected for the current batch. (3) *Drafting:* The EAGLE drafter takes the previous token and its hidden state from the base model, running draft model auto-regressively to construct a tree of draft tokens. (4) *Validation:* Here, the draft tokens are verified using the base model. The tree-attention-based inference enables efficient one-shot validation of the entire tree of draft tokens. (5) *Sampling:* This stage involves deciding whether to accept some or all of the proposed tokens. Following EAGLE, we use multi-round speculative sampling which preserves the output probability distribution of the original LLM model. (6) *Bookkeeping:* EAGLE speculative decoding requires caching hidden states for the next round of drafting. Therefore, during bookkeeping, the KV cache and hidden states are rewounded to the appropriate position based on the accepted length.

3.1 Tree attention

Tree attention (Miao et al., 2024) is an important technique in EAGLE speculative decoding, and is used in both drafting and verification stages. During each decoding round, one path in the tree is partially accepted based on an acceptance criterion, which utilizes validation logits generated by the target model at each node. A naive approach to compute these logits would involve unrolling all possible paths and performing a prefill computation with an effective batch size equal to the product of the batch size and the number of paths. However, a more efficient method would be to flatten all draft tokens into a single sequence. This approach

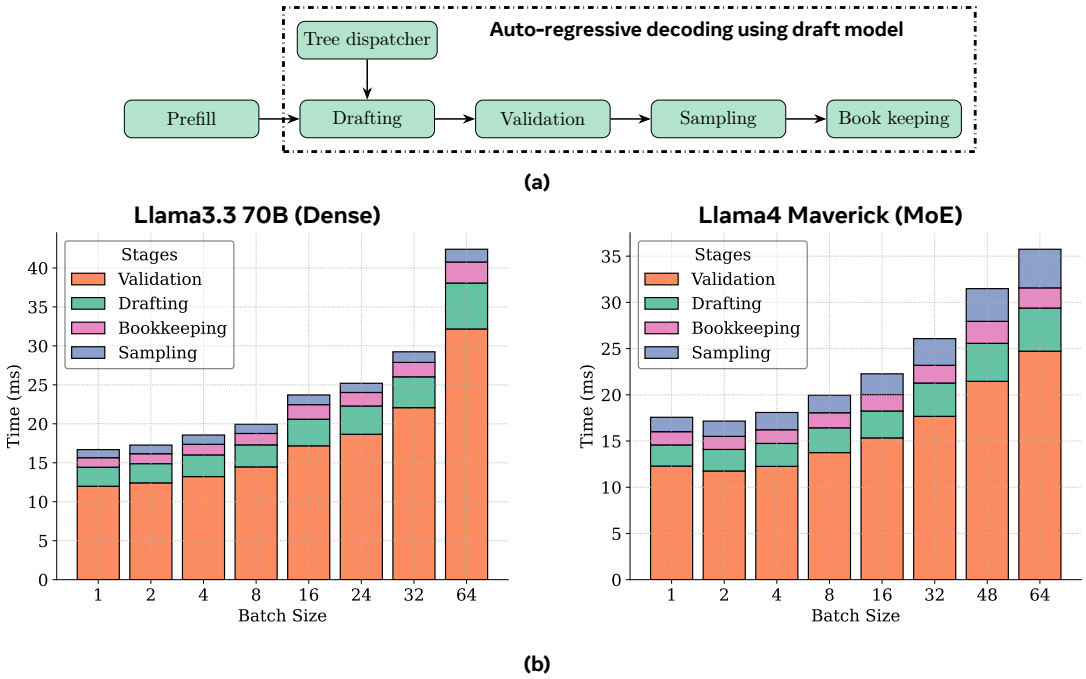


Figure 4 Inference Workflow. (a) Flow diagram illustrating EAGLE speculative decoding in our production environment. (b) Example showing a breakdown of the decoding step latency for each stage of the inference workflow at varying batch sizes, measured for Llama3.3 70B and Llama4 Maverick on an NVIDIA H100 GPU. Note that, in this example, three tokens are generated auto-regressively from the draft model and validated by the base model in a single decoding step. Because the decoding step generates multiple tokens, it should not be confused with TTIT, which measures the speed of decoding a single token.

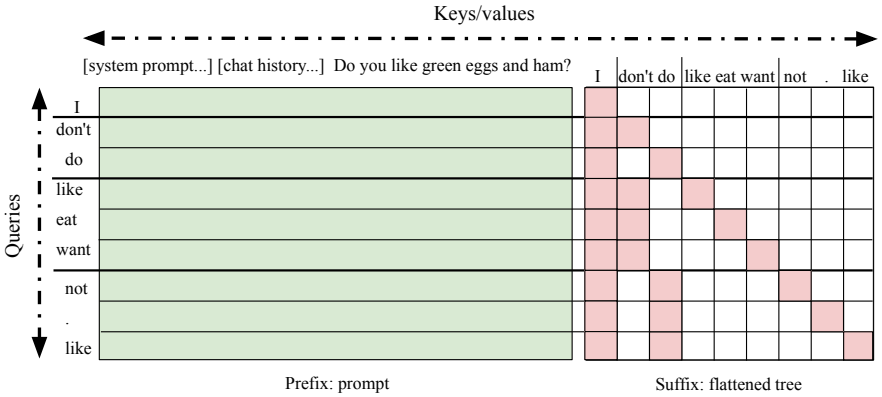


Figure 5 An illustration of optimized tree attention. Unlike the standard and unoptimized tree attention (see Figure 4 in Miao et al. (2024)), we split the attention operation into prefix and suffix for efficient inference.

introduces a challenge when computing attention, as the flattened tree tokens are not in the natural sequential order, making it hard to apply standard causal attention mask.

A custom tree mask can be applied by providing an explicit mask tensor. However, tree attention with an explicit mask is slow due to the large shape of the mask, which scales with the query and context (keys/values) length. Therefore, we split the attention computation into two parts: (a) attention between the query and context (prefix), and (b) attention within the query itself (suffix), as shown in Figure 5. The former is computationally expensive but mask-free, whereas the latter requires a tree mask but is relatively small. This two pass approach allows us to effectively prepare and extract query/key/values for speculated tokens needed for suffix calculations without any performance overhead. These two attention computations are

then aggregated using `merge_attentions`, a simple method which computes full attention based on partial attention outputs computed on two disjoint chunks of KV context (Juravsky et al., 2024). We have used this method to implement efficient tree attention in xFormers (Lefaudeux et al., 2022). It's worth mentioning that our optimized tree attention code in xFormers can be used seamlessly with other tree-based speculative decoding methods, such as SpecInfer (Miao et al., 2024) and Medusa (Cai et al., 2024), to further improve their inference efficiency.

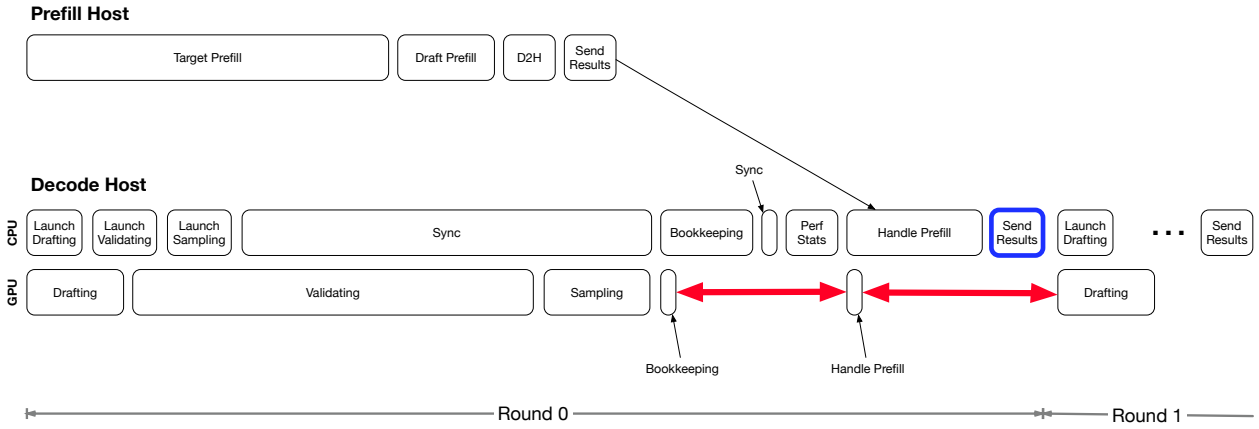
3.2 Multi-round speculative sampling

Multi-round speculative sampling (MSS) in EAGLE extends the standard speculative sampling method of Leviathan et al. (2023); Chen et al. (2023), specifically adapting it for tree-like drafts. Naive implementation of MSS can introduce significant overhead in production decoding environments due to the nested loops over tree depth and the need to launch numerous small kernels. To address this, we implemented the following optimizations:

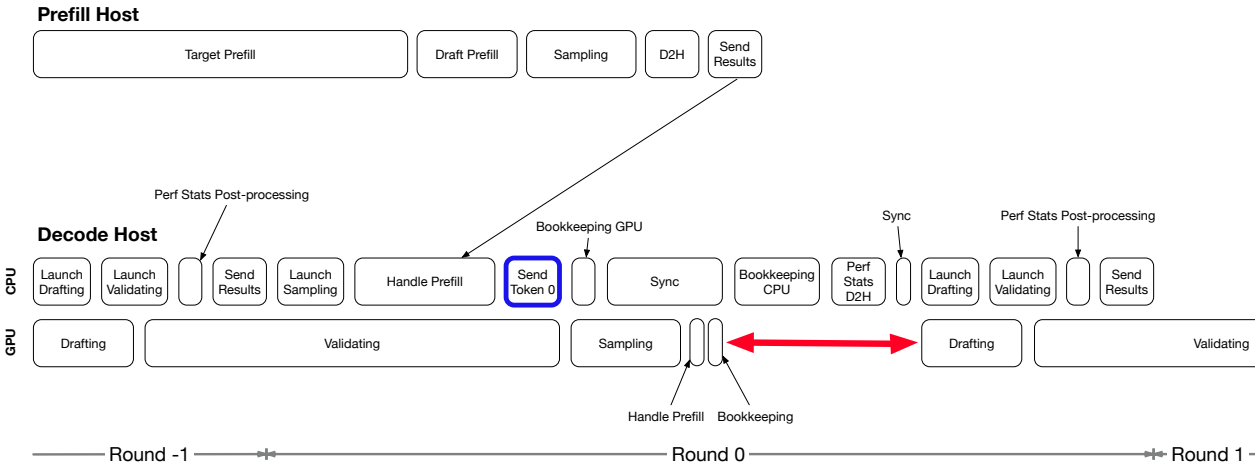
- ***PyTorch-2 compilation:*** While most operations in MSS are not inherently computationally intensive, the nested loop over tree depth and the children of each node can result in significant CPU overhead, particularly in large-scale production environments. To mitigate this, we compiled the code using PyTorch-2, achieving a 1.5× speedup. A key consideration was handling the dynamic batch dimension for variable incoming traffic. Naively applying `torch.compile` could lead to recompilation for each batch size, potentially causing latency spikes in production. To address this, we designated the batch dimension as dynamic in every input tensor. With this change, recompilations are limited to just two cases: batch size of one and batch sizes greater than one. Moreover, if a service receives warm-up traffic at startup, and both scenarios are covered, the compiler cache is populated by `torch.compile` during the warm-up phase. This cache is then reused during real traffic, helping to avoid latency spikes.
- ***Parallelisation across tensor parallel ranks:*** Despite the GPU-efficient implementation and PyTorch 2 compilation, sampling becomes a noticeable bottleneck in decoding performance at large batch sizes. One primary reason is the application of the top-p mask or nucleus sampling (Holtzman et al., 2020), which involves inherently "serial" operations that are challenging to accelerate on the GPU. Observing that (a) sampling is embarrassingly parallel over the batch dimension and (b) all tensor parallel (TP) ranks should receive the same sampling result, we parallelized the sampling across TP ranks by padding the batch size to a multiple of the total number of GPUs (aka, world size). However, with this simple solution, special care must be taken to synchronize random number generators across ranks. Indeed, during sampling, each rank generates random tensors with sizes proportional to the batch size. If different ranks process different local batch sizes, such as when the last rank handles the remainder of the global batch size divided by the world size, the random number generators may diverge, potentially causing system hangs in subsequent iterations. To prevent this, we generate a full-sized random tensor on each rank and have each rank take a different slice of this tensor.
- ***Greedy draft decoding:*** Different methods for sampling draft tokens are known for tree-shaped drafts, including multinomial (with and without replacement) and greedy sampling (Miao et al., 2024; Jeon et al., 2024). Specifically, for greedy decoding, we consider a node with k children, and place k tokens with the highest probabilities into these child nodes. In our experiments, we observed slightly higher TPC from greedy sampling compared to other methods. Moreover, the computationally expensive top-p mask is a no-op for draft probabilities with greedy decoding, and allows us to skip top-p mask at the drafting stage (while still applying it to target logits at the validation stage). This reduces the computational overhead from drafting and further helps in improving inference efficiency.

3.3 Disaggregated inference with large batch sizes

In production environments, we use a disaggregated approach, prefill and decode operations are performed on separate hosts (see Figure 6). In this setup, the client communicates with the decode host, which redirects requests to prefill hosts for the first iteration and handles the remaining iterations itself. Due to this disaggregation, speculative decoding must accommodate large batch sizes and variable traffic, which can be challenging as it complicates efficient computational resource management. Furthermore, the increased



(a) Before latency hiding optimizations



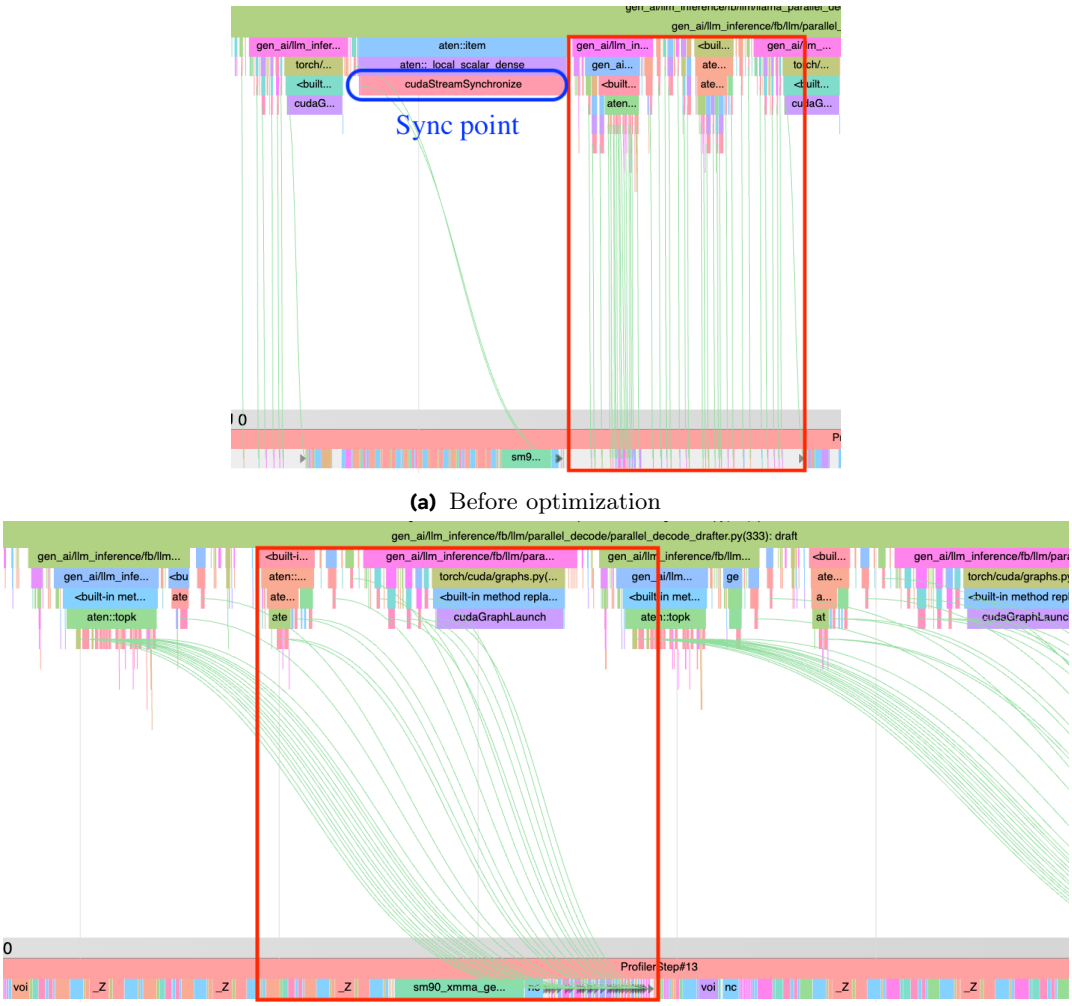
(b) After latency hiding optimizations

Figure 6 Disaggregated decoding cycle restructuring optimized inference efficiency by overlapping CPU and GPU tasks. This reduced GPU idle time and improved overall efficiency, as indicated by the red arrows in **(a)** and **(b)**.

demand for FLOPs for larger batch sizes requires careful allocation of hardware resources to ensure that the system can handle peak loads without bottlenecks, including out-of-memory errors.

To handle large batch-sizes with speculative decoding, we implemented the following optimizations that improve execution efficiency and GPU utilization to 94%:

- We collected hardware traces to identify unnecessary CPU-GPU synchronization points and removed them. This allows the CPU to run ahead of the GPU, ensuring that the GPU has sufficient tasks in its work queue. This helps reduce GPU idle time and improve latency. An example trace is shown in Figure 7. At the synchronization point highlighted by the blue box in Figure 7a, the CPU waited for GPU execution to finish, resulting in inefficiencies and slowdowns in subsequent operations due to CPU kernel launch overhead and GPU idleness, as indicated by the red box in Figure 7a. After removing this synchronization point, as shown in Figure 7b, CPU kernels were launched well ahead of GPU execution. This optimization eliminated GPU idle time and improved TTIT by 0.4ms. On an average, we found that TTIT was improved by 8-12% after removing all possible CPU-GPU synchronization points.
- We restructured the decoding cycle by overlapping CPU operations with GPU kernel execution to further improve latency. Specifically, we decomposed tasks (e.g., bookkeeping) into distinct GPU and CPU components. This allowed us to concurrently execute CPU post-processing tasks while GPU kernels are running, effectively hiding CPU operations behind GPU processing and improving overall system efficiency. As shown in Figure 6, there are gaps between bookkeeping, prefill handling, and drafting, as



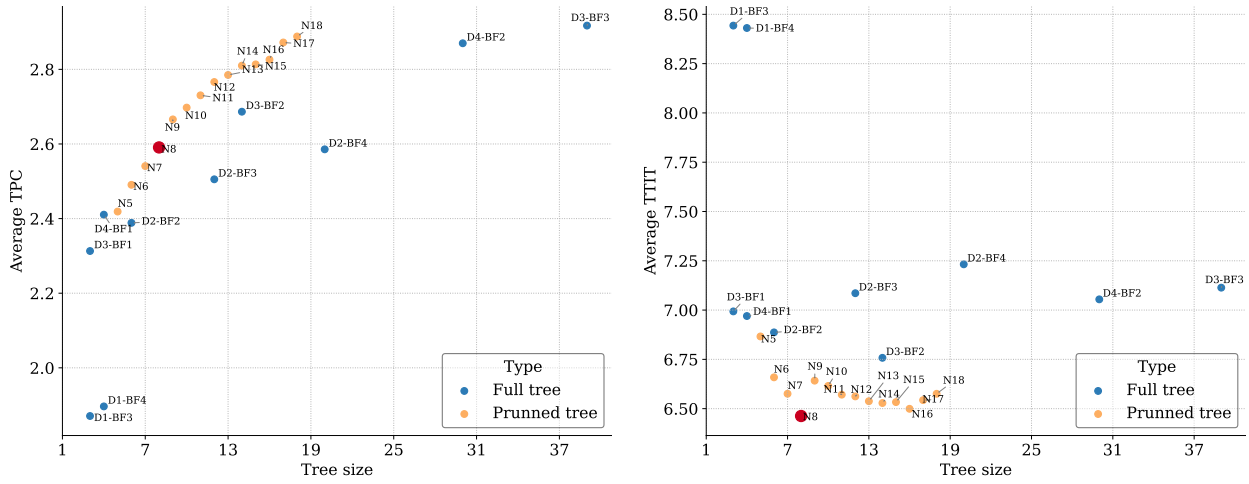
(a) Before optimization

(b) After optimization

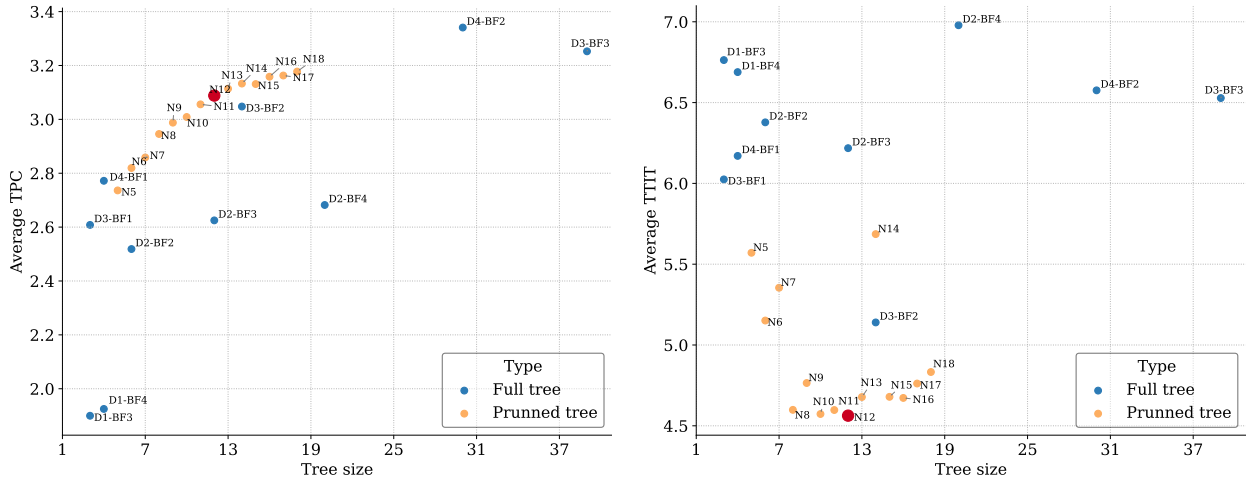
Figure 7 An example showing the effect of removing unnecessary CPU-GPU synchronization point from decoding flow. Initially, the CPU waited for GPU execution (highlighted as blue box in **(a)**), causing slowdowns (highlighted as red box in **(a)**). After removing synchronization points, CPU kernels launched ahead of GPU execution, reducing idle time (highlighted as red box in **(b)**).

highlighted by red arrows in Figure 6a. With restructuring, we were able to reduce the GPU idle time (see red arrow in Figure 6b), which helped in improving the TTIT on an average by about 10%.

- To optimize the time to first token (TTFT), we added a sampling step at the end of the prefill phase (indicated by the blue box in Figure 6b). This allows us to immediately consume the outputs (hidden state and next predicted token) of the first token as soon as they are received by the decoder, rather than waiting for the entire decoding cycle to complete. By doing so, we reduce latency and improve response time by approximately 8-30% on average, depending on the traffic volume.
- We reordered the processing sequence by moving prefill response handling ahead of bookkeeping, allowing us to hide it behind the long-running validation kernels. Additionally, we initiated the subsequent round of drafting and validating kernels earlier, effectively masking the latency associated with transmitting results to the client.



(a) Llama3.3 70B (Dense)



(b) Llama4 Scout (MoE)

Figure 8 Effect of different tree structures on TPC and TTIT for two different Llama models. We study two different tree structures: (1) a **full tree** with depth m and branching factor n, and is represented as $\text{D}_m\text{-BF}_n$ in graph, and (2) **prunned tree** with i nodes, and is represented as N_i in a graph. The optimal tree configurations that are used in our production environments for both models are different and highlighted in **red** (N8 and N12 for Llama3 70B and Llama4 Scout respectively). Here, TTIT is measured with a batch size of one on NVIDIA H100 GPUs. Here, TPC is measured with temperature=0, top-p=0.9, and speculation length of three.

3.4 Other performance optimizations

Pre-computed static trees. In our production environment, the batch size varies widely, ranging from 1 to over 100. This variability presents a challenge, as no single tree structure is optimal across the entire range. Figure 8 shows that larger tree structures (which are effective at small batch sizes) yield higher TPC but are slow. Furthermore, at larger batch sizes, the computational cost outweighs the benefits of increased TPC. To select the optimal tree configurations, we developed *tree dispatcher*. Based on a pre-computed static tree structures, it decides which tree configuration to use for the current batch size. For example, for a model typically running at small batch sizes we can decide to use a static tree when batch size is 1 and a chain draft of 3 tokens when batch size is larger than 1. This way we account for a trade-off between the computational cost and TPC, and ensure optimal performance across all traffic conditions.

Draft KV cache alignment At the beginning of each drafting stage, the EAGLE method overwrites the draft model's KV cache by executing a forward pass using the previous round's newly accepted tokens and hidden

states. Synchronizing the KV cache at this stage ensures that the draft model incorporates relevant contextual information from the previous validation round, resulting in more accurate and coherent drafting. However, this approach incurs a non-negligible additional computation cost. We found that using only the last output token and its corresponding hidden states from the previous round is sufficient to align the KV cache of the draft model with the base model while improving inference efficiency.

CUDA graphs. During inference, particularly with large batch sizes, the drafting and validation stages are the most computationally demanding due to model execution. We expand the usage of CUDA Graphs in Meta's internal inference engine to optimize drafting and validation. This effectively eliminates kernel launch overhead and streamlines GPU operations. Notably, we chose to capture the entire model execution in a CUDA graph for maximum performance. This approach contrasts with some open-source inference libraries (e.g., vLLM-v1 (Kwon et al., 2023)), which separate attention computation from CUDA graph capture to offer greater flexibility.

Attention kernel Attention used in the decoding process of LLMs is bottlenecked by the need to load keys and values from the cache located in GPU high-bandwidth memory (HBM). As a result, the runtime scales linearly with the context length. There has been several efforts to implement attention efficiently on a given hardware (e.g., Dao et al., 2023, 2022; Dao, 2024).

For faster decoding in production environments, we use hardware-specific attention kernels. In our environment, we have three options: (a) Flash Decoding (or Triton Split-k kernel), (b) Flash Attention v2 and v3, and (c) Composable Kernel on AMD (ROCm, 2023). Based on the batch size, tree size, and GPU type, we use a heuristic-based method to select among these attention options, which is primarily derived from the KV split strategy in these approaches. For example, the flash decoding kernel outperforms the others in terms of latency when the tree size is ≤ 4 and the batch size is ≤ 64. Flash attention v3 provides the best latency for larger trees and batch sizes when supported by the hardware.

Paged KV with tree attention Paged KV partitions the KV cache into blocks and allocate the blocks as needed (Kwon et al., 2023). We used paged KV to optimize memory usage, and maintain separate caches for base and draft models. However, paged KV is not compatible with tree attention. To make it compatible, we implemented two changes: (a) the suffix part in tree attention is relatively small. To minimize the overhead of looking up paged blocks, we only apply paging to the prefix attention, and write back the intermediate keys and values instead of retrieving them from the cache, and (b) we add additional tree padding when allocating blocks for drafting and validation stages to prevent speculative tokens from exceeding block boundaries.

Persistent KV Our system leverages persistent KV caching to enhance cross-request performance. As requests are processed, least recently used (LRU) blocks from the paged KV cache are evicted to the persistent KV cache in order to accommodate new KVs. Then, during prefill, these cached KVs are reused for matching prompt prefixes, thereby reducing time to first token (TTFT).

We maintain separate persistent KV caches for base and draft models, ensuring that both can benefit from this optimization. Additionally, our system enforces that both models show similar cache hit and block eviction behaviors for any given sequence of tokens. This was done primarily to simplify the implementation and improve maintainability. To achieve this uniform behavior, we made the following adjustments:

- We prefill both the base and draft model KV caches with the entire sequence of prompt tokens. However, for drafting, EAGLE requires the hidden state of the previous token from the base model, which is not available for the first prompt token. Unlike EAGLE, which skips the first prompt token during draft prefill, we fuse the first token with its own hidden state outputted from the base model prefill. Empirically, we have observed that this approach results in a slight increase in TPC.
- To maintain synchronization between the draft and base models, we scale the draft model's KV cache size by a factor of $\frac{\text{number of draft layers}}{\text{number of base layers}}$ relative to the base model's KV cache size. This ensures that both the paged and persistent caches of both models have identical sets of blocks, leading to synchronized block eviction times.

	TPC ↑			Drafting Latency (ms) ↓		
	BF16	FP8	INT4	BF16	FP8	INT4
Llama3.1 8B (Dense)	2.79	2.79	2.76	1.00	0.93	0.96
Llama3.3 70B (Dense)	2.95	2.95	2.96	1.00	0.89	0.83
Llama4 Scout (MoE)	2.86	2.86	2.87	1.00	0.89	0.87
Llama4 Maverick (MoE)	2.81	2.79	2.78	1.00	0.93	0.92

Table 2 Effect of FFN quantization in draft models. Here, TPC is measured with chain-like draft, temperature=0, and top-p=0.9 on MT-Bench. Also, drafting latency is the time to generate three tokens (corresponding to speculation length in our experiments) auto-regressively from the draft model.

Quantized draft model The "losslessness" aspect of speculative decoding is that the draft model's quality does not affect the final output; it only impacts the speed. Therefore, we can compress the draft model differently than the base model. Table 2 shows that INT4 feed-forward network quantization provides a good trade-off between TPC and decoding speed.

Guided decoding Guided decoding helps generate structured outputs and is crucial in production environments (Willard and Louf, 2023). To enable speculative decoding for guided decoding requests, we implemented two changes: (a) We integrated guided decoding logic across all stages of the speculative decoding workflow, including drafting, sampling, and bookkeeping. (b) We improved performance by optimizing GD-related operations on the GPU. Specifically, we efficiently initialized the GD finite state machine (FSM) states and accelerated the key step of masking logits by moving CPU-related operations to the GPU. This avoided synchronization overhead, and we observed a speed-up of 2.6× when guided decoding with speculative decoding was used compared to the non-speculative decoding baseline. We note that results presented in this paper are without guided decoding.

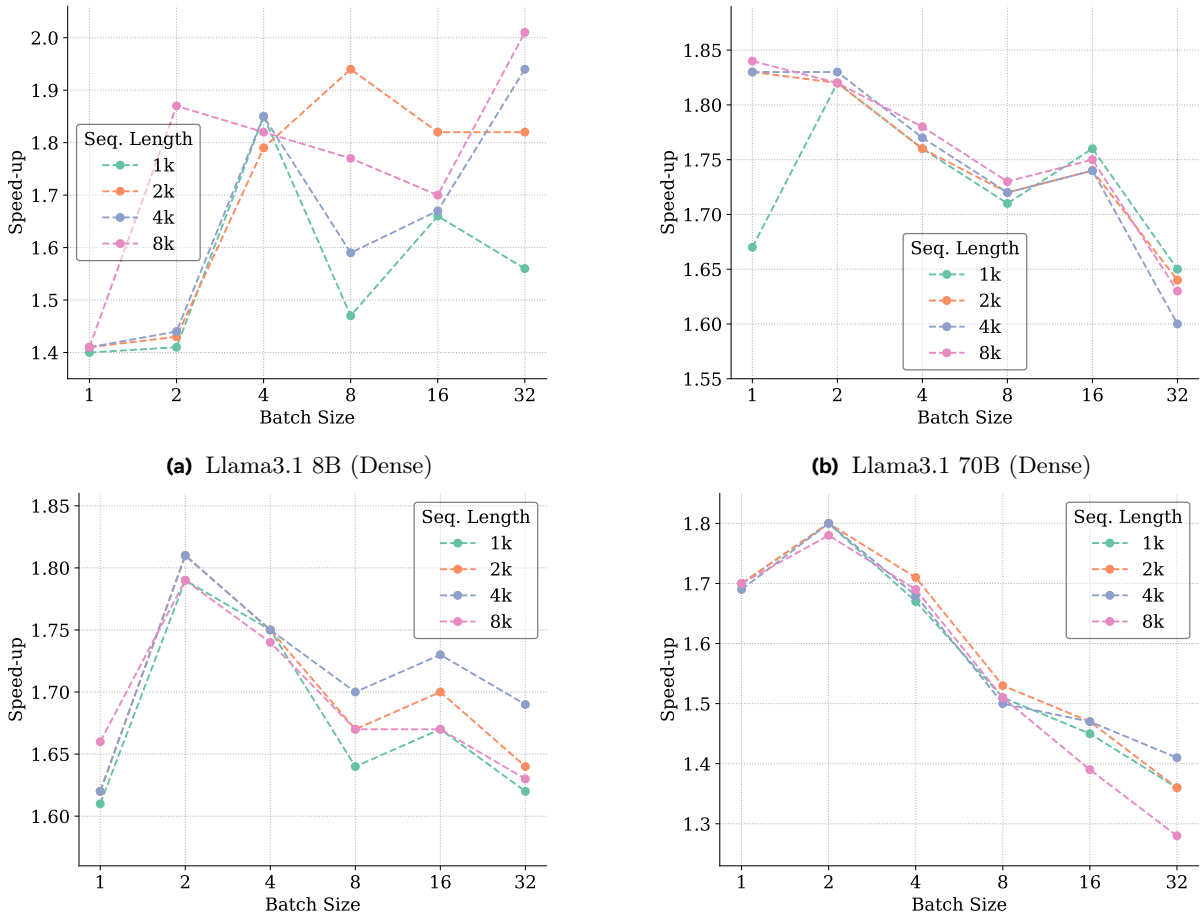
iRoPE for Llama4 Llama4 introduced interleaved ROPE (iRoPE), a variant of local attention where sequences are split into sub-sequences of fixed length, and queries can only attend to values within the same sub-sequence. This effectively modifies the attention mask, making the blocks in the block-diagonal mask smaller. Recall that tree attention (see Section 3.1) also modifies the attention mask. Therefore, it is essential to combine these two modifications correctly for Llama4 inference.

The iRoPE attention mask for decoding was implemented using XFormers' gappy attention biases. We adapted the tree attention to use XFormers' gappy attention biases in prefix attention and integrated iRoPE logic through the validation path. One corner case that we encountered was when the draft string crosses the boundary of two sub-sequences. While it is theoretically possible to implement complex logic for splitting the tree mask between two sub-sequences, we opted for a simpler solution: truncating the draft to fit entirely within one sub-sequence. As such situations are extremely rare, the impact on the acceptance rate is negligible.

3.5 Benchmarking

Many previous works assumed that speculative decoding speed-up decreases as batch size increases (Su et al., 2023; Miao et al., 2024; Liu et al., 2024). This is likely because speculative decoding methods may be under-utilizing GPU *computational resources*, i.e., increased FLOPs from running drafting and validation comes for free because decoding is bound by the GPU memory bandwidth and not compute. However, at large batch sizes, decoding becomes compute-bound, resulting in a decrease in speedup.

At large context lengths, attention dominates the computation even for large batch sizes, and therefore the workload stays memory-bound (Sadhukhan et al., 2025). Our results in Figure 9 partially confirm these observations. Overall, we observe that the relationship between end-to-end speculative decoding speed-up and batch size varies depending on the model size and its performance characteristics. For instance, the Llama3.1 8B model exhibits greater speculative decoding speedup at large batch sizes compared to small batch sizes. In contrast, the speed-up for Llama4 Maverick, which has approximately 400 billion parameters, decreases with increasing batch size.



(a) Llama3.1 8B (Dense)

(b) Llama3.1 70B (Dense)

(c) Llama4 Scout (MoE)

(d) Llama4 Maverick (MoE)

Figure 9 Inference speed-up of various Llama models using EAGLE speculative decoding, measured relative to the baseline performance. The plot shows how speed-up varies with different batch sizes and sequence lengths.

Contributors

This project is the result of efforts by numerous individuals within the GenAI and Infra teams at Meta. Below, we acknowledge all core contributors and contributors, listed in alphabetical order by first name.

Core contributors. Bangsheng Tang, Carl Chengyan Fu, Fei Kou, Grigory Sizov, Haoci Zhang, Jason Park, Jiawen Liu, Jie You, Qirui Yang, Sachin Mehta, Shengyong Cai, Xiaodong Wang, Xingyu Liu, Yunlu Li, Yanjun Zhou, Wei Wei, Zhiwei Zhao, and Zixi Qi.

Contributors. Adolfo Victoria, Aya Ibrahim, Bram Wasti, Changkyu Kim, Daniel Haziza, Fei Sun, Giancarlo Delfin, Emily Guo[†][1], Jialin Ouyang, Jaewon Lee, Jianyu Huang, Jeremy Reizenstein, Lu Fang, Quinn Zhu, Ria Verma[†], Vlad Mihailescu, Xingwen Guo, Yan Cui, Ye Hu, and Yejin Lee.

Acknowledgments

We thank Chuanhao Zhuge, Emad El-Haraty, Lai Wei, Mohammad Rastegari[†], Rajasi Saha, Seiji Yamamoto, Sergey Edunov, Shaun Lindsay, Sijia Chen, and Tony Liu for their support. We also thank Edward Yang who helped to make `torch.compile` work well on multiround speculative sampling. We also thank the broader GenAI and Infra team members for the discussions and feedback.

[1] † Work done while working at Meta.

References

Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

Mandeep Baines, Shruti Bhosale, Vittorio Caggiano, Naman Goyal, Siddharth Goyal, Myle Ott, Benjamin Lefaudeux, Vitaliy Liptchinsky, Mike Rabbat, Sam Sheiffer, et al. Fairscale: A general purpose modular pytorch library for high performance and large scale training, 2021.

Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Tri Dao, Daniel Haziza, Francisco Massa, and Grigory Sizov. Flash-decoding for long-context inference. https://crfm.stanford.edu/2023/10/12/flashdecoding.html, October 12 2023. Stanford University.

Aaron Grattafiori et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020. https://openreview.net/forum?id=rygGQyrFvH.

Wonseok Jeon, Mukul Gagrani, Raghavv Goel, Junyoung Park, Mingu Lee, and Christopher Lott. Recursive speculative decoding: Accelerating llm inference via sampling without replacement. *arXiv preprint arXiv:2402.14160*, 2024.

Jordan Juravsky, Bradley Brown, Ryan Ehrlich, Daniel Y Fu, Christopher Ré, and Azalia Mirhoseini. Hydragen: High-throughput llm inference with shared prefixes. *arXiv preprint arXiv:2402.05099*, 2024.

Diederik P Kingma. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

Woosuk Kwon et al. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

Benjamin Lefaudeux et al. xformers: A modular and hackable transformer modelling library. https://github.com/facebookresearch/xformers, 2022. The tree attention implementation can be found at https://github.com/facebookresearch/xformers/blob/8fc8ec5a4d6498ff81c0c418b89bbaf133ae3a44/xformers/ops/tree_attention.py.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pages 19274–19286. PMLR, 2023.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint arXiv:2401.15077*, 2024.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840*, 2025.

Xiaoxuan Liu, Cade Daniel, Langxiang Hu, Woosuk Kwon, Zhuohan Li, Xiangxi Mo, Alvin Cheung, Zhijie Deng, Ion Stoica, and Hao Zhang. Optimizing speculative decoding for serving large language models using goodput, 2024. https://arxiv.org/abs/2406.14066.

Meta. LLaMA 4: Multimodal Intelligence. https://ai.meta.com/blog/llama-4-multimodal-intelligence/, 2025. Accessed: 2025-06-05.

Xupeng Miao et al. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pages 932–949, 2024. Figures are referred from this version of the paper: https://arxiv.org/abs/2305.09781.

Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: A kvcache-centric disaggregated architecture for llm serving. *URL https://arxiv. org/abs/2407.00079*, 2024.

Markus N Rabe and Charles Staats. Self-attention does not need $o(n^2)$ memory. *arXiv preprint arXiv:2112.05682*, 2021.

AMD ROCm. Composable kernel. https://github.com/ROCm/composable_kernel, 2023. GitHub repository.

Ranajoy Sadhukhan, Jian Chen, Zhuoming Chen, Vashisth Tiwari, Ruihang Lai, Jinyuan Shi, Ian En-Hsu Yen, Avner May, Tianqi Chen, and Beidi Chen. Magicdec: Breaking the latency-throughput tradeoff for long context generation with speculative decoding, 2025. https://arxiv.org/abs/2408.11049.

Project SGLang. Sglang: A language for sgl project. https://github.com/sgl-project/sglang, 2023. Accessed: 2025-06-03.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.

Qidong Su, Christina Giannoula, and Gennady Pekhimenko. The synergy of speculative decoding and batching in serving large language models, 2023. https://arxiv.org/abs/2310.18813.

Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Brandon T Willard and Rémi Louf. Efficient guided generation for llms. *arXiv preprint arXiv:2307.09702*, 2023.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36:46595–46623, 2023.

Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210, 2024.